

TaskFlow

Actas de Reunión de Equipo

Sistema de Gestión de Proyectos y Tareas

Integrantes del Equipo

Jota López	Desarrollo Backend / Arquitectura
Esmeralda Rivas Guzmán	Desarrollo Frontend / Diseño UI

Período cubierto: Marzo – Junio 2026

Metodología: Scrum (sprints semanales)

Total de reuniones documentadas: 10

Espacio: *Espacio del equipo [ES-LA]*

1	Índice
1. Fase de Gestión y Análisis (Marzo 2026)	2
1.1. Reunión 1 — Kick-off y Levantamiento de Requisitos	2
1.2. Reunión 2 — Revisión Sprint 1 y Planificación Sprint 2	3
1.3. Reunión 3 — Revisión Sprints 2-3 y Cierre de Gestión	4
2. Fase de Diseño (Abril – Mayo 2026)	5
2.1. Reunión 4 — Inicio de Diseño: Wireframes y Prototipos	5
2.2. Reunión 5 — Arquitectura del Sistema y Configuración Docker	6
2.3. Reunión 6 — Modelo Entidad-Relación y Arranque de Desarrollo	8
3. Fase de Desarrollo (Mayo – Junio 2026)	9
3.1. Reunión 7 — Revisión Sprint 1: Autenticación y Gestión de Usuarios	9
3.2. Reunión 8 — Revisión Sprint 2: Proyectos, Tareas y Kanban	11
3.3. Reunión 9 — Revisión Sprint 3: Comentarios, Adjuntos y Dashboards	12
3.4. Reunión 10 — Cierre del Proyecto: Pruebas, Despliegue y Retrospectiva	14
4. Resumen Ejecutivo del Proyecto	17

1

Fase

de Gestión y Análisis (Marzo 2026)

1 Reunión 1 — Kick-off y Levantamiento de Requisitos

Fecha:	Lunes, 2 de marzo de 2026
Hora:	10:00 – 11:15 h
Modalidad:	Videollamada (Google Meet)
Asistentes:	Jota López, Esmeralda Rivas Guzmán
Sprint ref.:	Gestión y Análisis — Sprint 1 (3/2 – 3/15)

Agenda

1. Definición del alcance del proyecto TaskFlow.
2. Identificación de requisitos funcionales y no funcionales.
3. Asignación inicial de responsabilidades.
4. Elaboración del Diccionario EDT.

Discusión

Jota: Bueno, empecemos. Ya tenemos claro que el proyecto es un sistema de gestión de proyectos y tareas orientado a equipos pequeños. Lo que necesitamos definir hoy es qué funcionalidades son obligatorias para el MVP y cuáles son opcionales.

Esmeralda: Estoy de acuerdo. Creo que lo más crítico es tener autenticación con roles, la gestión de proyectos y tareas, y algún tipo de tablero Kanban. Sin eso no tiene sentido el sistema.

Jota: Exacto. Yo añadiría también el control de acceso por rol desde el inicio, porque si lo dejamos para después, tenemos que refactorizar toda la API. Es mucho mejor diseñarlo bien desde el comienzo.

Esmeralda: Sí, tiene sentido. ¿Y qué ponemos como requisitos no funcionales? Yo creo que deberíamos incluir que la interfaz sea responsiva, que el tiempo de respuesta de la API sea razonable, y que el sistema soporte Docker para el despliegue.

Jota: Perfecto. Agrego que la autenticación debe usar JWT con expiración configurable, y que las contraseñas deben guardarse con hash. También deberíamos mencionar que el sistema tiene que ser mantenible, así que vamos a documentar la arquitectura desde el principio.

Esmeralda: De acuerdo. Respecto al Diccionario EDT, ¿lo dividimos por fases del proyecto o por módulos funcionales?

Jota: Yo prefiero por fases: Gestión, Diseño y Desarrollo. Así queda alineado con cómo lo tenemos en ClickUp y es más fácil de hacer seguimiento.

Esmeralda: Me parece bien. Yo me encargo de estructurar el documento del Diccionario EDT esta semana y tú revisas los requisitos que voy levantando, ¿te parece?

Jota: Perfecto. Yo en paralelo empiezo a pensar en la arquitectura técnica, aunque eso lo formalizamos en la fase de Diseño.

Acuerdos:

- Esmeralda redacta el Formato Diccionario EDT y el documento de Requisitos Funcionales y No Funcionales.
- Jota revisa y valida ambos documentos antes del 15 de marzo.
- El MVP incluye: autenticación JWT con roles, gestión de proyectos, gestión de tareas y tablero Kanban.
- El sistema será contenerizado con Docker desde el inicio.

1Reunión 2 — Revisión Sprint 1 y Planificación Sprint 2

Fecha:	Lunes, 16 de marzo de 2026
Hora:	09:30 – 10:30 h
Modalidad:	Videollamada (Google Meet)
Asistentes:	Jota López, Esmeralda Rivas Guzmán
Sprint ref.:	Gestión y Análisis — Sprint 2 (16/3 – 22/3)

Agenda

1. Revisión de entregables del Sprint 1: Requisitos y EDT.
2. Planificación del Diagrama de Gantt.
3. Identificación de hitos clave del proyecto.

Discusión

Esmeralda: Terminé el Diccionario EDT y el documento de requisitos. Los subí al repositorio. Quedaron bastante completos, aunque tuve que tomar algunas decisiones sobre los requisitos no funcionales sin consultarte, espero que estén bien.

Jota: Los revisé esta mañana. Están bien estructurados. Solo añadiría como requisito no funcional que el sistema debe manejar correctamente los errores de la API con códigos HTTP apropiados, pero lo podemos ajustar ahora sin problema. El EDT me parece que cubre bien las tres fases.

Esmeralda: Bien, lo agrego. Ahora para el Gantt, ¿usamos alguna herramienta específica o lo hacemos directamente en el documento?

Jota: Yo creo que lo más práctico es hacerlo en una herramienta visual y luego exportarlo. Así podemos actualizar si cambian las fechas. ¿Tienes experiencia con alguna?

Esmeralda: Puedo usar GanttProject o simplemente Google Sheets con formato visual. Lo importante es que refleje bien las dependencias entre fases.

Jota: Sí, las dependencias son clave. La fase de Desarrollo no puede empezar hasta que el Diseño esté listo, y el Diseño depende de que los requisitos estén aprobados. Eso tiene que quedar claro en el Gantt.

Esmeralda: Perfecto. También hay que incluir los sprints de desarrollo con sus fechas tentativas. Según lo que tenemos en ClickUp, el desarrollo empieza el 11 de mayo, así que tenemos tiempo.

Jota: Correcto. Dejemos las fechas de desarrollo como estimaciones por ahora. Lo importante esta semana es tener el Gantt completo con todas las fases.

Acuerdos:

- Ambos elaboran el Diagrama de Gantt para entrega el 22 de marzo.
- El Gantt debe reflejar las dependencias entre las tres fases del proyecto.
- Jota ajusta el documento de requisitos con el punto adicional sobre manejo de errores HTTP.

1 Reunión 3 — Revisión Sprints 2-3 y Cierre de Gestión

Fecha:	Lunes, 30 de marzo de 2026
Hora:	10:00 – 11:00 h
Modalidad:	Videollamada (Google Meet)
Asistentes:	Jota López, Esmeralda Rivas Guzmán
Sprint ref.:	Gestión y Análisis — Sprint 4 (30/3 – 5/4)

Agenda

1. Revisión del Diagrama de Gantt y Registro de Riesgos.
2. Consolidación de entregables de planificación.
3. Cierre formal de la fase de Gestión.

Discusión

Jota: El Gantt quedó bien. Refleja correctamente las tres fases y las dependencias. Lo que me parece que faltó definir mejor son los criterios de aceptación por sprint en

Desarrollo.

Esmeralda: Sí, eso lo añadimos al Registro de Riesgos como un riesgo de alcance. También identifiqué el riesgo de que el equipo es de solo dos personas, lo cual puede generar cuellos de botella si una persona se bloquea.

Jota: Buen punto. ¿Y cómo mitigamos ese riesgo?

Esmeralda: Documentando bien el trabajo de cada uno, haciendo revisiones cruzadas de código y dejando comentarios claros. Si uno se bloquea, el otro puede retomar la tarea.

Jota: De acuerdo. Otro riesgo que veo es la integración frontend-backend. Si trabajamos en silos y no probamos la integración frecuentemente, podemos llegar al final con problemas difíciles de resolver.

Esmeralda: Totalmente. Para eso propongo que en cada sprint de Desarrollo hagamos al menos una sesión de integración y prueba conjunta antes del cierre del sprint.

Jota: Me parece excelente. ¿Agregamos eso al documento de entregables de planificación como buena práctica del equipo?

Esmeralda: Sí, lo agrego. Creo que con el Registro de Riesgos y los entregables de planificación, cerramos bien la fase de Gestión.

Acuerdos:

- Se cierra oficialmente la Fase de Gestión y Análisis.
- Se documenta como buena práctica: una sesión de integración por sprint de Desarrollo.
- El Registro de Riesgos incluye: cuellos de botella por equipo reducido e integración frontend-backend tardía.
- La próxima reunión inicia la Fase de Diseño (15 de abril).

1

Fase

de Diseño (Abril – Mayo 2026)

1Reunión 4 — Inicio de Diseño: Wireframes y Prototipos

Fecha:	Martes, 15 de abril de 2026
Hora:	10:00 – 11:00 h
Modalidad:	Videollamada (Google Meet)
Asistentes:	Jota López, Esmeralda Rivas Guzmán
Sprint ref.:	Diseño — Sprint 1 (4/15 – 4/21)

Agenda

1. División de tareas: wireframes vs. prototipos de alta fidelidad.
2. Definición de paleta de colores y sistema de diseño.
3. Pantallas prioritarias a diseñar en el sprint.

Discusión

Esmeralda: Para los wireframes, propongo usar Miro. Es más rápido para hacer bocetos y puedo compartirlo fácilmente contigo para que des retroalimentación en tiempo real.

Jota: Me parece bien. Yo mientras tanto me encargo de los prototipos en Figma una vez que tengamos los wireframes base aprobados, así los prototipos ya tienen una estructura clara detrás.

Esmeralda: Perfecto. ¿Por qué pantalla empezamos? Yo diría que la más crítica es el login, luego el listado de proyectos, y el tablero Kanban.

Jota: Coincido. Agrego el panel de gestión de usuarios para administradores, porque ahí es donde se ve el rol diferenciado. Si eso no queda claro en el diseño, en desarrollo vamos a tener problemas.

Esmeralda: Sí, tiene razón. También tenemos que pensar en la navegación general: sidebar o topbar, si usamos modales o páginas separadas para las formularios...

Jota: Yo prefiero sidebar fijo con topbar para el usuario activo. Y para formularios de creación y edición, modales. Así no rompemos el flujo del usuario.

Esmeralda: De acuerdo con los modales. Para el sidebar, ¿qué secciones ponemos?

Jota: Como mínimo: Dashboard, Proyectos, y si hay tiempo, Reportes. Para la versión MVP eso es suficiente.

Esmeralda: Listo, empiezo hoy con los wireframes de esas pantallas en Miro y te notifico cuando estén listos para revisión.

Acuerdos:

- Esmeralda desarrolla wireframes en Miro (login, listado proyectos, Kanban, gestión usuarios).
- Jota desarrolla prototipos de alta fidelidad en Figma basados en los wireframes.
- Diseño general: sidebar fijo, topbar con info de usuario, formularios en modal.
- Entrega esperada: 21 de abril.

1Reunión 5 — Arquitectura del Sistema y Configuración Docker

Fecha:	Martes, 22 de abril de 2026
Hora:	10:00 – 11:30 h
Modalidad:	Videollamada (Google Meet)
Asistentes:	Jota López, Esmeralda Rivas Guzmán
Sprint ref.:	Diseño — Sprint 2 (4/22 – 4/28)

Agenda

1. Revisión de wireframes y prototipos del Sprint 1.
2. Definición de la arquitectura técnica del sistema.
3. Planificación de la configuración Docker.
4. Inicialización de los proyectos frontend y backend.

Discusión

Jota: Los wireframes en Miro están muy bien. Me gustó especialmente la vista del tablero Kanban con las columnas diferenciadas. Los prototipos en Figma los hice basándome en eso y quedaron bastante parecidos. Creo que ya podemos arrancar con código.

Esmeralda: Bien. ¿Y qué stack definimos? Ya habíamos hablado de React y Node, pero ¿confirmamos las versiones y librerías?

Jota: Para el backend: Node con Express y Prisma como ORM. PostgreSQL como base de datos. Para el frontend: React con Vite por el rendimiento en desarrollo, y TailwindCSS para no tener que escribir CSS desde cero.

Esmeralda: Me parece perfecto. Yo ya tengo experiencia con Vite y Tailwind, así que eso me ayuda. ¿Y la autenticación?

Jota: JWT. Yo me encargo de toda la lógica del backend: generación del token, middleware de autenticación y middleware de autorización por rol. Tú del lado del frontend haces el AuthContext en React y las rutas protegidas.

Esmeralda: Perfecto, así nos dividimos bien el trabajo de autenticación. ¿Y Docker?

Jota: Hacemos tres servicios en el docker-compose: frontend, backend y PostgreSQL. Yo hago el Dockerfile del backend y el docker-compose completo con las dependencias entre servicios. Tú haces el Dockerfile del frontend.

Esmeralda: Entendido. ¿Volúmenes persistentes para la base de datos?

Jota: Sí, imprescindible. Si no, perdemos datos cada vez que bajamos los contenedores. Lo incluyo en la configuración.

Esmeralda: También deberíamos implementar la ruta `/api/health` desde el principio para poder verificar que el backend responde antes de empezar con las funcionalidades.

Jota: Buena idea. Eso también nos sirve para el healthcheck de Docker Compose. Lo añadimos al sprint.

Acuerdos:

- Stack confirmado: React + Vite + Tailwind (frontend), Node + Express + Prisma + PostgreSQL (backend).
- Jota: Dockerfile backend, docker-compose.yml con tres servicios y volúmenes, schema inicial de Prisma.
- Esmeralda: Dockerfile frontend, inicializar proyecto React + Vite + Tailwind, ruta `/api/health`, README.
- Jota documenta la arquitectura en diagrama de capas y el flujo de autenticación JWT.

1Reunión 6 — Modelo Entidad-Relación y Arranque de Desarrollo

Fecha:	Martes, 29 de abril de 2026
Hora:	09:30 – 10:45 h
Modalidad:	Videollamada (Google Meet)
Asistentes:	Jota López, Esmeralda Rivas Guzmán
Sprint ref.:	Diseño — Sprint 3 (4/29 – 5/5) / Previo Desarrollo Sprint 1

Agenda

1. Revisión de la arquitectura y la configuración Docker.
2. Diseño del modelo entidad-relación de la base de datos.
3. Alineación previa al inicio del desarrollo.

Discusión

Esmeralda: La configuración Docker funcionó bien en mi equipo. El frontend conecta correctamente con el backend por la red interna de Docker. La ruta `/api/health` responde 200. Podemos decir que el entorno está listo.

Jota: Perfecto. Por mi parte, el diagrama de arquitectura ya documenta bien las tres capas y el flujo JWT. Ahora lo que necesitamos es tener claro el modelo de datos antes de empezar a codear, porque cambiar el esquema de Prisma a mitad del desarrollo es costoso.

Esmeralda: Totalmente de acuerdo. ¿Cuáles son las entidades principales que identificas?

Jota: User, Project, Phase, Task, Comment y ProjectMember para la relación muchos a muchos entre usuarios y proyectos. Eso cubre el MVP completo.

Esmeralda: Yo añadiría un campo para el archivo adjunto en Comment, porque según los requisitos queremos soportar adjuntos en comentarios.

Jota: Cierto, lo había olvidado. Podemos agregar `attachmentUrl` y `attachmentName` en la entidad Comment. Eso es suficiente para el MVP sin complicar demasiado el modelo.

Esmeralda: Me encargo de identificar todas las entidades a partir de los requisitos funcionales, definir los atributos y tipos de dato, establecer las relaciones y cardinalidades, y luego diagramar el ER en la herramienta que usamos.

Jota: Y yo hago la revisión final del diagrama para asegurar que esté alineado con lo que voy a implementar en Prisma. ¿Te parece si lo tenemos listo para el 5 de mayo?

Esmeralda: Sí, eso me da tiempo suficiente. Cuando lo tenga, te lo comparto para revisión antes de la fecha límite.

Jota: Perfecto. Con eso cerramos Diseño y el 11 de mayo arrancamos Desarrollo al 100 %.

Acuerdos:

- Esmeralda diseña el diagrama ER completo (entidades, atributos, relaciones) para el 5 de mayo.
- Jota realiza la revisión final del ER y da el visto bueno antes del inicio del desarrollo.
- Entidades definidas: User, Project, Phase, Task, Comment (con adjuntos), ProjectMember.
- Inicio oficial del desarrollo: 11 de mayo de 2026.

1

Fase

de Desarrollo (Mayo – Junio 2026)

1Reunión 7 — Revisión Sprint 1: Autenticación y Gestión de Usuarios

Fecha:	Sábado, 17 de mayo de 2026
Hora:	10:00 – 11:15 h
Modalidad:	Videollamada (Google Meet)
Asistentes:	Jota López, Esmeralda Rivas Guzmán
Sprint ref.:	Desarrollo — Sprint 1 (5/11 – 5/17)

Agenda

1. Revisión del Sprint 1 de Desarrollo: autenticación y usuarios.
2. Demo de funcionalidades implementadas.
3. Identificación de deuda técnica.
4. Planificación del Sprint 2.

Discusión

Jota: Terminé el backend de autenticación. El endpoint `POST /api/auth/login` valida credenciales, genera el JWT firmado y lo retorna. El middleware de autenticación verifica el token en cada request protegido, y el middleware de autorización revisa el rol. Puedes hacer las pruebas con el usuario admin que sembré en la base de datos.

Esmeralda: Perfecto. Yo implementé el lado del frontend: la pantalla de login en React, el AuthContext que guarda el token y el usuario, las rutas protegidas con React Router, y también el botón de cierre de sesión en el navbar. Todo funciona bien con el backend de Jota.

Jota: Excelente. ¿Algún problema que encontraste al integrar?

Esmeralda: Tuve un problema menor con los headers CORS al principio, pero lo resolví configurando el middleware de CORS en el backend para aceptar el origen del frontend. También noté que cuando el token expira, el frontend no redirige automáticamente al login, simplemente falla silenciosamente.

Jota: Buen punto. Tengo que añadir un interceptor en el cliente HTTP del frontend que detecte el 401 y haga logout automático. Lo agrego como deuda técnica para este sprint o el siguiente.

Esmeralda: Yo puedo implementar eso en el AuthContext. Lo hago esta semana. También terminé los endpoints de gestión de usuarios: `POST /api/users` con hash de contraseña, `GET /api/users`, y Jota hizo el `PATCH` y `DELETE`.

Jota: Sí, implementé el `DELETE /api/users/:id` con validación para que un usuario no pueda eliminarse a sí mismo, y el `PATCH` para actualizar datos. Del lado del frontend, hice la pantalla de gestión de usuarios para admin con el formulario modal de creación.

Esmeralda: Y yo hice los modales de edición de rol y confirmación de eliminación, y los botones en la tabla. Creo que el módulo de usuarios está bastante completo.

Acuerdos:

- Sprint 1 cerrado con éxito: autenticación JWT, cierre de sesión y CRUD de usuarios completos.
- Esmeralda implementa interceptor HTTP para redirect automático al expirar el token.

- El Sprint 2 inicia el 18 de mayo con foco en gestión de proyectos y tablero Kanban.

Deuda técnica registrada:

- Interceptor HTTP para manejo automático de expiración del JWT (401 → logout).

1Reunión 8 — Revisión Sprint 2: Proyectos, Tareas y Kanban

Fecha:	Domingo, 24 de mayo de 2026
Hora:	10:00 – 12:00 h
Modalidad:	Videollamada (Google Meet)
Asistentes:	Jota López, Esmeralda Rivas Guzmán
Sprint ref.:	Desarrollo — Sprint 2 (5/18 – 5/24)

Agenda

1. Demo del módulo de proyectos.
2. Demo del tablero Kanban y movimiento de tareas.
3. Revisión de asignación de responsables y registro de tiempo.
4. Planificación Sprint 3.

Discusión

Jota: Este sprint fue el más denso hasta ahora. Del lado del backend, implementé todos los endpoints de proyectos: **POST**, **GET**, **PATCH** y **DELETE**, además del endpoint de creación de fase personalizada y los de asociación y eliminación de miembros. También los endpoints de cambio de fase de tarea, asignación de responsable y registro de tiempo real con acumulación.

Esmeralda: Y yo del frontend implementé toda la UI correspondiente: el listado de proyectos, los formularios de creación y edición con validación de fechas, los modales de confirmación, el panel de gestión del equipo, el tablero Kanban con columnas, las tarjetas de tarea, el movimiento de tareas entre fases, la identificación visual de tareas vencidas, el selector de ejecutor y el formulario de registro de tiempo.

Jota: Fue un sprint muy productivo. ¿Tuviste algún problema con el Drag & Drop del Kanban?

Esmeralda: Decidí no usar Drag & Drop por el momento para no añadir una dependencia más. En su lugar, implementé botones de mover a la fase anterior/siguiente, que funcionan bien y es más accesible. Si hay tiempo al final, lo mejoramos.

Jota: Me parece una decisión sensata. La funcionalidad es la misma y no dependemos de librerías externas. También noté que la tarjeta de tarea vencida tiene un borde rojo en el frontend, eso está bien.

Esmeralda: Sí, y el backend ya calcula si la tarea está vencida basándose en la fecha de vencimiento y el estado. Así el frontend solo consume esa información.

Jota: Exacto, mejor tener esa lógica en el backend para que sea consistente. ¿Algo pendiente de este sprint?

Esmeralda: El formulario de asignación de responsable todavía no filtra los usuarios por los que pertenecen al proyecto. Actualmente muestra todos los usuarios del sistema. Eso hay que corregirlo.

Jota: Tienes razón. Yo ya implementé el filtro en el backend, el endpoint retorna solo los miembros del proyecto. El problema debe estar en cómo el frontend construye la petición. Lo revisamos juntos después de la reunión.

Acuerdos:

- Sprint 2 cerrado: proyectos, tareas, Kanban, fases personalizadas y registro de tiempo completos.
- Jota y Esmeralda corrigen juntos el filtro de miembros en el selector de responsable.
- Sprint 3 (25-31 mayo): comentarios en tareas, adjuntos, dashboards de carga y avance.

Corrección urgente:

- Selector de responsable de tarea debe filtrar solo miembros del proyecto actual.

1Reunión 9 — Revisión Sprint 3: Comentarios, Adjuntos y Dashboards

Fecha:	Sábado, 31 de mayo de 2026
Hora:	10:30 – 12:00 h
Modalidad:	Videollamada (Google Meet)
Asistentes:	Jota López, Esmeralda Rivas Guzmán
Sprint ref.:	Desarrollo — Sprint 3 (5/25 – 5/31)

Agenda

1. Demo del módulo de comentarios con adjuntos.
2. Demo de los dashboards de carga laboral y avance del proyecto.
3. Revisión de la corrección del filtro de miembros.

4. Planificación del Sprint 4 (cierre).

Discusión

Jota: Lo primero: la corrección del filtro de miembros quedó bien. Era un problema en el frontend, como sospechaba.

Esmeralda: Sí, no estaba pasando el ID del proyecto en la petición. Ya está corregido y funciona perfecto. Ahora solo aparecen los miembros del proyecto en el selector.

Jota: Sobre los comentarios: implementé los endpoints de creación y listado. También configuré multer para la carga de archivos en el backend, el endpoint de carga con validación de formato y tamaño, y el almacenamiento en volumen Docker para que los archivos persistan.

Esmeralda: Y yo diseñé e implementé la sección de comentarios en la vista de detalle de tarea, incluyendo el selector de tipo de interacción con etiquetas visuales, el componente de carga de archivos en el formulario de comentario, y la visualización y descarga de adjuntos. Quedó bastante bien.

Jota: Lo vi y sí, la interfaz de comentarios es limpia. Los dashboards, ¿cómo quedaron?

Esmeralda: Implementé los tres componentes: el dashboard principal con resumen del proyecto, la gráfica de progreso de tareas por fase, y la gráfica comparativa de tiempo estimado vs. real. Consumen los endpoints que tú hiciste.

Jota: Exacto, implementé el endpoint de resumen general del proyecto y el endpoint de carga laboral por usuario. La carga laboral calcula el número de tareas activas y el tiempo estimado pendiente por persona.

Esmeralda: Funciona bien. Hay un caso borde que notamos: cuando un usuario no tiene tareas asignadas no aparece en el dashboard de carga laboral. ¿Es un bug o un comportamiento esperado?

Jota: Es un comportamiento esperado por ahora. Si no tiene tareas, no tiene carga. Pero si queremos mostrar a todos los miembros aunque su carga sea cero, lo podemos ajustar en el Sprint 4. Depende del tiempo.

Esmeralda: Lo añado como mejora opcional para el Sprint 4. Para el sprint final también queda la ejecución de pruebas integrales, el despliegue y la verificación de accesibilidad en los dashboards.

Jota: Sí. El Sprint 4 es básicamente cierre y calidad. Tenemos que asegurarnos de que todo el flujo funciona de punta a punta y que el despliegue con Docker es limpio.

Acuerdos:

- Sprint 3 cerrado: comentarios con adjuntos y dashboards de carga y avance completos.
- Corrección previa pendiente resuelta: filtro de miembros funcionando.

- Sprint 4 (1-7 junio): pruebas integrales, despliegue y verificación de accesibilidad.
- Mejora opcional Sprint 4: mostrar todos los miembros en dashboard de carga, incluso con carga cero.

1 Reunión 10 — Cierre del Proyecto: Pruebas, Despliegue y Retrospectiva

Fecha:	Domingo, 7 de junio de 2026
Hora:	10:00 – 12:30 h
Modalidad:	Videollamada (Google Meet)
Asistentes:	Jota López, Esmeralda Rivas Guzmán
Sprint ref.:	Desarrollo — Sprint 4 (6/1 – 6/7) — Reunión de Cierre

Agenda

1. Resultados de las pruebas integrales.
2. Verificación de accesibilidad en dashboards.
3. Demo final del sistema completo.
4. Retrospectiva del proyecto.
5. Cierre oficial.

Discusión

Jota: Las pruebas integrales pasaron en su mayoría. Probé todos los flujos críticos: registro de usuario, login, creación de proyecto, asignación de miembros, creación de tareas, movimiento en Kanban, comentarios con adjuntos y los dashboards. Solo encontré un problema: el endpoint de carga de archivos falla cuando se sube un archivo mayor a 5 MB con un mensaje de error poco informativo.

Esmeralda: Lo vi también. Mejoré el mensaje de error del frontend para que diga explícitamente el límite de tamaño. ¿El backend ya tiene la validación?

Jota: Sí, multer rechaza el archivo, pero el error llega como un 500 genérico. Cambia a un 413 con un mensaje claro. Lo corrijo ahora mismo antes de que cerremos el sprint.

Esmeralda: Sobre la accesibilidad: revisé los dashboards con las herramientas de accesibilidad del navegador. Los colores de las gráficas pasan el contraste WCAG AA. También agregué `aria-label` a los elementos interactivos del tablero Kanban y las gráficas.

Jota: Excelente. El despliegue también fue exitoso. Levanté todo con `docker-compose up` en una máquina limpia y funcionó a la primera, que era nuestra prueba de fuego. El volumen de PostgreSQL persiste los datos correctamente entre reinicios.

Esmeralda: Eso es una gran victoria. Que funcione en máquina limpia significa que el sistema está bien encapsulado.

Jota: Exactamente. Ahora la retrospectiva. ¿Qué fue lo que funcionó mejor del proyecto?

Esmeralda: Creo que la división del trabajo fue muy clara desde el inicio: yo frontend y tú backend, pero con revisiones cruzadas constantes. Eso evitó que nos piéramos en el detalle del otro y al mismo tiempo mantuvimos coherencia.

Jota: Totalmente. También funcionó bien el haber definido la arquitectura y el modelo de datos antes de empezar a codear. Me ahorró mucho tiempo de refactoring. ¿Qué mejorarías?

Esmeralda: Creo que podríamos haber implementado pruebas unitarias desde el principio, no al final. Tuvimos que escribirlas a posteriori y algunas partes de la lógica no estaban bien desacopladas para testear fácilmente.

Jota: Muy de acuerdo. Para el próximo proyecto: TDD o al menos pruebas en paralelo al desarrollo. También creo que podríamos haber usado ramas de feature en Git más consistentemente, a veces commiteamos directo a **develop**.

Esmeralda: Sí, la disciplina de Git es algo que tenemos que mejorar. Pero en general estoy muy satisfecha con el resultado. El sistema es funcional, tiene buena UX y está bien estructurado.

Jota: Completamente de acuerdo. TaskFlow está listo para entregarse.

Acuerdos finales:

- Jota corrige el código de error del endpoint de carga de archivos (500 → 413 con mensaje claro).
- Sistema desplegado y verificado en entorno limpio con Docker.
- Verificación de accesibilidad WCAG AA completada en todos los dashboards.
- **Proyecto TaskFlow cerrado exitosamente el 7 de junio de 2026.**

1 Retrospectiva del Proyecto

Categoría	Observación
Logros	División clara frontend/backend, arquitectura definida antes del desarrollo, Docker funcional en máquina limpia, dashboards accesibles.
Mejoras	Adoptar TDD o pruebas en paralelo al desarrollo; mayor disciplina en ramas de Git (<i>feature branches</i>).
Próximos pasos	Documentar lecciones aprendidas en el README; aplicar prácticas de Git flow en futuros proyectos.

1

Ejecutivo del Proyecto

Resumen

lightblue Sprint	Entregables principales
Gestión S1 (3/2–3/15)	Requisitos funcionales/no funcionales, Diccionario EDT
Gestión S2 (16/3–22/3)	Diagrama de Gantt
Gestión S3 (23/3–29/3)	Registro de riesgos
Gestión S4 (30/3–5/4)	Entregables de planificación y control
lightgreen Diseño S1 (4/15–4/21)	Wireframes en Miro, Prototipos en Figma
lightgreen Diseño S2 (4/22–4/28)	Arquitectura del sistema, entorno Docker, inicialización del proyecto
lightgreen Diseño S3 (4/29–5/5)	Modelo Entidad-Relación
lightorange Desarrollo S1 (5/11–5/17)	Autenticación JWT, gestión de usuarios (CRUD)
lightorange Desarrollo S2 (5/18–5/24)	Proyectos, tareas, Kanban, fases, responsables, tiempo
lightorange Desarrollo S3 (5/25–5/31)	Comentarios con adjuntos, dashboards de carga y avance
lightorange Desarrollo S4 (6/1–6/7)	Pruebas integrales, despliegue, accesibilidad, cierre

Cuadro 1: Resumen de sprints y entregables del proyecto TaskFlow

lightblue Integrante	Rol principal	Responsabilidades clave
Jota López	Backend / Arquitectura	API REST (Express), JWT, Prisma, Docker Compose, endpoints de lógica de negocio, dashboards backend
Esmeralda Rivas G.	Frontend / Diseño	React + Vite + Tailwind, wireframes, prototipos Figma, UI completa, AuthContext, componentes Kanban y dashboards

Cuadro 2: Distribución de responsabilidades